

RAGFailBench: Automated Construction and Controllable Failure-Injection Benchmarking for Retrieval-Augmented Generation

Anonymous Author(s)

Affiliation omitted for double-blind review

Abstract

Most RAG benchmarks are released once and go stale as the underlying knowledge base changes; RAGFailBench instead automatically regenerates a failure-aware benchmark whenever the knowledge base changes. Retrieval-Augmented Generation (RAG) systems are increasingly deployed in enterprise knowledge applications, yet existing RAG benchmarks are largely static datasets that cannot continuously evaluate evolving knowledge bases or simulate realistic retrieval failures. We present RAGFailBench, an automated framework for constructing validated benchmark datasets and generating controlled retrieval failures for regression testing of enterprise RAG systems. The framework automatically builds provenance-preserving question-answering (QA) seeds from a live corpus, validates generated samples through multi-stage filtering (rule-based checks, LLM-judge review, and baseline-model screening), injects six parameterized Failure Operators under a matched context budget, and evaluates model robustness under controlled conditions. On a frozen 1,000-page pilot corpus (768 pages after quality filtering; 12,366 chunks), three independent seeded runs show a stable automatic construction funnel (QA acceptance $39.2\% \pm 0.2\%$; failure-verification pass rate $97.5\% \pm 0.1\%$) and human-audited quality metrics that are stable across seeds (clean-seed acceptance $66.0\% \pm 1.8\%$; stratified failure validity $96.8\% \pm 1.2\%$). Evaluating three production-grade models across the six failure categories reveals sharply distinct robustness profiles: all three models almost always abstain when evidence is absent, remain near ceiling under context noise, chunk-boundary splitting, and evidence-position perturbation, and diverge most under conflicting evidence, where one model's accuracy drop and abstention rate roughly double those of the other two. RAGFailBench contributes (1) an automated, reproducible benchmark-construction pipeline, (2) a validated, parameterized failure-injection framework, and (3) a continuous benchmark-regeneration and regression-testing workflow that connects knowledge-base updates to deployment decisions for enterprise RAG systems.

Keywords—retrieval-augmented generation, benchmark construction, regression testing, failure injection, enterprise AI, LLM evaluation, reproducibility

1. Introduction

Enterprise knowledge bases change continuously. Confluence pages are edited, SharePoint libraries are reorganized, internal wikis are updated after incidents, product documentation is revised with every release, and

policy documents are amended on a regulatory cadence. Each such update triggers, at minimum, a re-indexing step and a redeployment of the retrieval-augmented generation (RAG) system that serves answers over that content. In most production teams we have observed, this deploy cycle is validated against a small, manually curated “golden” QA regression set that was written once and rarely revisited. That practice misses the majority of realistic failure modes a RAG system encounters in production: evidence that has been deleted or moved, context windows diluted with near-duplicate pages, answers split across chunk boundaries by an indexing job, evidence buried in the middle of a long context, documents that contradict each other after a partial rollout, and near-miss retrieved passages that read as relevant but do not contain the answer.

Existing RAG benchmarks do not close this gap. Foundational RAG work and the surveys that followed it establish the modeling paradigm [1–4], but the resulting benchmarks are largely static, manually or semi-automatically curated question sets that are frozen at release time. Evaluation frameworks such as RAGAS, ARES, and RGB score end-to-end pipeline output on such fixed data [6–8], and benchmark suites such as RAGBench and CRUD-RAG extend coverage but remain point-in-time snapshots [9,10]. None of these are designed to be regenerated against a changing corpus, and none expose a controllable, typed taxonomy of retrieval failures with tunable severity. As a result, teams operating enterprise RAG systems lack a way to answer a concrete, recurring question: after this week's document changes, did retrieval robustness regress, and against which failure mode specifically?

We address this gap with RAGFailBench, an automated framework for (i) constructing validated, provenance-preserving benchmark datasets directly from a live document corpus, and (ii) injecting six parameterized failure operators into those datasets to produce controlled, labeled retrieval-failure conditions for regression testing. The pipeline moves from raw documents to a regenerable evaluation report: documents are ingested and chunked, candidate QA pairs are generated and passed through multi-stage validation to produce clean seeds, six failure operators transform clean seeds into typed failure cases under a matched context budget, and models are evaluated on both clean and failure conditions so that accuracy drops can be attributed to a specific failure type rather than reported only as an aggregate score.

We evaluate the framework on a frozen pilot corpus of 1,000 target English Wikipedia pages (768 pages after

quality filtering; 12,366 chunks) [50], chosen for open provenance and independent verifiability during development, and report three seeded end-to-end runs to characterize the stability of the automatic construction pipeline together with human-audited quality of both clean seeds and injected failures. We then evaluate three production-grade language models (referred to as nova-pro, llama, and gpt-oss) on the resulting benchmark and analyze their robustness profile across the six failure categories.

Contributions

C1 — Automated benchmark construction pipeline.

An end-to-end, configuration-driven pipeline that ingests a live document source via the MediaWiki API [51], generates candidate QA pairs with supporting-sentence provenance, and filters them through rule-based, LLM-judge, and baseline-model validation stages to produce a stratified set of clean seeds, with stability characterized across independent seeded runs.

C2 — Validated failure-generation framework with six controlled operators.

A typed, parameterized Failure Operator API (missing_evidence, context_noise, chunk_boundary, evidence_position, conflict, hard_negative) that transforms a clean seed into a labeled failure case under a matched context budget, with post-injection structural verification and stratified human validation.

C3 — Regression-testing workflow for enterprise RAG systems.

A workflow that automatically regenerates a failure-labeled benchmark whenever the knowledge base changes, tying corpus updates to a fresh, stratified seed pool, all six failure conditions, and a per-operator robustness report, positioning RAGFailBench as a continuous-integration artifact for RAG deployments rather than a one-time academic benchmark.

Research Questions

RQ1: How much do systems degrade under each Failure Operator? RQ2: Do models differ in their sensitivity to specific operators?

2. Related Work

2.1 Retrieval-Augmented Generation

Lewis et al. introduced Retrieval-Augmented Generation as a general recipe for conditioning sequence generation on retrieved documents, combining a parametric generator with a non-parametric retrieval index [1]. Ram et al. showed that in-context retrieval augmentation is

effective even without modifying the underlying language model [5]. Subsequent surveys chart the rapid expansion of this paradigm: Gao et al. organize the RAG literature around naive, advanced, and modular architectures [2]; Zhao et al. extend the survey lens to AI-generated content beyond text [3]; and Huang and Huang focus on retrieval-augmented text generation techniques specifically for large language models [4]. RAGFailBench is agnostic to the specific RAG architecture under test and instead targets the evaluation layer that sits above any such pipeline.

2.2 RAG Evaluation

A parallel line of work evaluates RAG pipeline quality rather than proposing new architectures. RAGAS computes reference-free metrics such as faithfulness and answer relevance directly from a pipeline's own retrieved context and generated answer [6]. ARES trains lightweight classifiers on synthetic data to automate the same style of judgment at lower cost [7]. RGB constructs a benchmark specifically to probe noise robustness, negative rejection, information integration, and counterfactual robustness in RAG systems [8]. RAGBench and CRUD-RAG extend coverage to explainable, multi-domain scoring and to knowledge-updating scenarios, respectively [9,10]. These systems are important precedents for the idea that RAG failure modes deserve dedicated measurement, and RGB in particular anticipates several of the stressors RAGFailBench operationalizes. The distinction we draw is architectural: RAGAS, ARES, RGB, RAGBench, and CRUD-RAG all evaluate models against a dataset that is fixed at release time. RAGFailBench instead treats the dataset itself as regenerable and treats each failure condition as a typed, parameterized object with a matched-budget clean counterpart, so a benchmark can be rebuilt automatically whenever the underlying corpus changes and degradation can be attributed to a specific, reproducible operator rather than an unlabeled drop in aggregate accuracy.

2.3 QA Dataset Construction

Manually curated QA datasets such as SQuAD [16], Natural Questions [17], TriviaQA [18], and HotpotQA [19] established the extractive and multi-hop QA formats that most RAG evaluation still relies on, and KILT unified several such tasks under a common Wikipedia-grounded retrieval setting [20]. These datasets were built once by large annotation efforts and are not designed to be regenerated as source documents change. RAGFailBench borrows the provenance-preserving, supporting-sentence style of QA construction popularized by this line of work but automates it end-to-end with LLM-based generation

and multi-stage validation, trading a fixed human-annotation budget for a pipeline that can be rerun against a new corpus snapshot.

2.4 Information Retrieval

Classic sparse retrieval (BM25 [21]) and dense retrieval methods — DPR [22], Contriever [23], and ColBERT / ColBERTv2 [24,25] — define the retrieval layer that RAG systems depend on, and BEIR provides a heterogeneous zero-shot benchmark across many such retrievers and domains [29]. RAGFailBench does not propose a new retriever; instead, several of its failure operators (notably `hard_negative` and `context_noise`) are designed to stress exactly the failure modes these retrieval benchmarks reveal in isolation — near-miss passages and topical noise — but do so inside a full generation pipeline rather than at the retrieval-metric level alone.

2.5 Benchmark Frameworks

General-purpose evaluation frameworks such as HELM [30], MLPerf Inference [31], and MTEB [32] establish methodology for holistic, multi-metric, multi-task benchmarking of language and embedding models respectively. RAGFailBench adopts a similar holistic-reporting ethos — accuracy, abstention, and per-operator drop, rather than a single aggregate score — but is scoped specifically to the RAG failure-diagnosis setting rather than general model capability.

2.6 Enterprise AI Evaluation

Industry tooling such as LangSmith [43], OpenAI Evals [44], DeepEval [45], and Promptfoo [46] provide infrastructure for tracing, scoring, and regression-testing LLM applications in production settings. These tools give teams a place to run evaluations but do not themselves generate failure-labeled benchmark data from a changing corpus; they assume the test set already exists. RAGFailBench is complementary to this tooling layer: it produces the regenerable, failure-typed dataset that such platforms can then execute against on a recurring, CI-triggered basis, in the spirit of continuous-integration and regression-test-selection practice from software engineering [47,48].

Taken together, no prior system across Sections 2.1–2.6 provides automated construction of failure-aware regression benchmarks for enterprise RAG: existing evaluation frameworks assume a fixed dataset, existing QA datasets are not regenerable against a changing corpus, and existing enterprise tooling assumes the benchmark already exists. RAGFailBench is positioned at the intersection of these three gaps.

2.7 Positioning Summary

Table 1. Capability comparison against representative prior RAG evaluation systems.

System	Automatic construction	Typed failure injection	Corpus regeneration	CI / regression-ready
RAGAS [6]	–	–	–	partial (metrics only)
ARES [7]	partial (synthetic training data)	–	–	partial (metrics only)
RGB [8]	partial	partial (noise, negative rejection)	–	–
RAGBench [9]	✓ (multi-domain)	–	–	–
CRUD-RAG [10]	✓	partial (update scenarios)	–	–
RAGFailBench (this work)	✓	✓ (6 typed, parameterized operators)	✓ (regenerable from live corpus)	✓ (per-operator regression report)

Table 1 summarizes the gap this paper targets. RGB is the closest prior system in spirit — it evaluates noise robustness, negative rejection, and counterfactual robustness — but its stress conditions are fixed at release rather than regenerable, parameterized operators with matched-budget clean counterparts. RAGBench and CRUD-RAG automate parts of dataset construction but do not expose a typed failure taxonomy, and none of the five prior systems ties construction to a corpus that can be refreshed on a recurring, CI-triggered basis.

3. Framework

RAGFailBench is organized as a linear pipeline: a live document corpus is filtered and chunked; candidate QA pairs are generated over those chunks; candidates pass through multi-stage validation to produce clean seeds; clean seeds are transformed by six Failure Operators into labeled failure cases under a matched context budget; and both clean and failure cases are scored under a shared evaluation protocol. Architecturally, this corresponds to eight modules in the released codebase: sources (corpus ingest), processing (filter, clean, deduplicate, chunk), generation (candidate QA), validation (rules, judge, baseline, seed selection), failures (operators and structural verification), evaluation (metrics and runner), reporting (report and human-review export), and experiments (multi-seed stability). Section 3.4 and Figure 1 detail how the six Failure Operators map onto specific stages of this pipeline, which is the basis for treating them as a principled taxonomy rather than an arbitrary list.

3.1 Corpus Construction

The pilot corpus is retrieved through the live MediaWiki API rather than a dated dump, which simplifies iteration during development but means the pilot fetch is not tied to a fixed revision snapshot; a pinned-revision or archival-dump ingest path is planned for a camera-ready release so that reported corpus statistics are exactly reproducible. Retrieved pages are filtered to remove disambiguation and stub pages, deduplicated, and split with section-aware chunking that respects heading boundaries rather than a fixed token window, so that a single chunk does not span two unrelated sections of a page. The pilot corpus used throughout this paper targets 1,000 English-language pages (200 per category) spanning five categories — person, location, science/technology, historical event, and organization/product — of which 768 pages survive quality filtering (200 person, 200 historical event, 141 location, 116 organization/product, 111 science/technology), yielding 12,366 chunks. The ingest module is written as a pluggable source adapter; Section 6.4 discusses what is required to point it at enterprise connectors (Confluence, SharePoint, PDF policy stores) instead of Wikipedia.

3.2 QA Generation

Candidate QA pairs are generated per chunk by an instruction-tuned LLM (Qwen2.5-7B-Instruct in the pilot configuration) prompted to produce a question, an answer, and the specific supporting sentence(s) in the source chunk that justify the answer. Retaining this provenance link at generation time is what later allows the `missing_evidence` and `hard_negative` operators to remove exactly the sentences that justify the answer, rather than removing content heuristically. Generation is run against a difficulty quota (40% easy / 40% medium / 20% hard by default) so that the resulting seed pool has a controlled difficulty mix rather than reflecting whatever distribution the source text happens to produce.

3.3 Validation

Each candidate QA pair is validated by a rule-first, then-parallel process rather than a strict three-stage sequential funnel. Candidates first pass deterministic rule checks, including a filter that rejects questions whose surface form leaks the page title (`title_leak_in_question`) and containment/uniqueness heuristics. Rule-passing candidates are then independently assessed by an LLM answerability judge, which reviews faithfulness to the supporting sentence, and a baseline-answer test, which runs a reference model against the question with only the gold chunk in context and rejects items the baseline model

itself cannot answer correctly; critically, the baseline check does not wait for the judge to accept a candidate, so judge and baseline are parallel, complementary signals rather than a chained gate. These signals are combined into a quality score; candidates below the `min_quality_score` threshold are rejected, and the accepted set is finally deduplicated. Items that survive this process are pooled and stratified-sampled into the final clean-seed set (200 seeds per run in the pilot configuration, drawn from 1,000 candidates). This validation process is intentionally strict: as reported in Section 5, roughly 39% of candidates are accepted, yet the resulting clean-seed pool still has a human-audited acceptance rate of only roughly 66%, indicating that automatic filtering, while strict on raw candidates, does not by itself guarantee human-level seed quality — a substantial share of automatically accepted items are still rejected on human inspection. Section 5.3 reports a validation component contribution analysis attributing rejection volume to each automatic signal, computed directly from stored per-candidate validation records without re-running any model; Section 6.1 discusses what that analysis implies about the acceptance-HAR gap and why human review remains a required stage of the pipeline rather than a formality.

3.4 Failure Operators

Failure Operators are the framework's core mechanism for turning a single clean seed into six labeled, controllable stress conditions. Each operator takes a clean seed as input and returns a failure case with the same question and gold answer but a rewritten context; each operator declares an expected model behavior (answer or abstain) and is instantiated at one of three severities (low, medium, high) that map to operator-specific parameters such as noise ratio, split depth, conflict prominence, or negative count. Every failure case is constructed under the same context chunk budget as its clean counterpart (8 chunks in the pilot), so that accuracy differences reflect the failure condition rather than a confound of context length. After injection, a structural verification pass checks each case for defects such as answer leakage in a case that should require abstention, and quarantines any case that fails this check; Section 5 reports a $97.5\% \pm 0.1\%$ pass rate for this automatic verification across three seeds, and a $96.8\% \pm 1.2\%$ human-audited validity rate on stratified samples of the surviving cases.

Table 2. The six Failure Operators, their pipeline stage, mechanism, and expected model behavior.

Operator	Stage	Mechanism	Expected behavior
missing_evidence	evidence / retrieval	Remove supporting sentence(s) / gold chunk, leaving only distractors	Abstain
hard_negative	retrieval	Replace gold chunk with topical near-miss passages only	Abstain
conflict	knowledge state	Keep gold chunk; insert a contradictory alternate-claim passage	Answer (prefer gold)
context_noise	context assembly	Keep gold chunk; add distractor chunks at a controlled noise ratio	Answer
chunk_boundary	chunking	Split the supporting sentence across an artificial chunk cut	Answer
evidence_position	context assembly	Move the gold chunk to the front, middle, or end of the context window	Answer

quality independent of content. We view this as the operator set's current coverage rather than a closed taxonomy: extending it (e.g., `citation_error`, `long_context`, as noted in the released repository) means adding an operator to an existing stage or, if a new pipeline stage is identified, adding a new stage column to Figure 1.

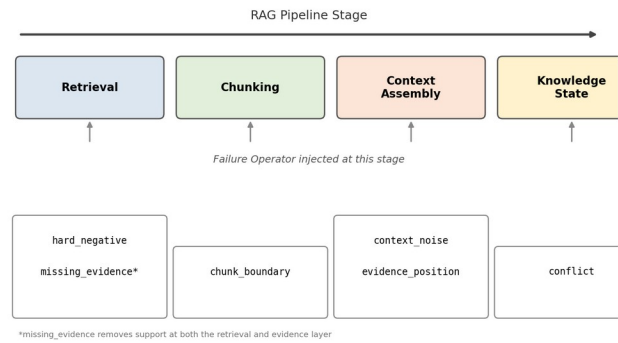


Figure 1. Failure Operator taxonomy: each operator is anchored to the specific RAG pipeline stage it corrupts (retrieval, chunking, context assembly, or knowledge state), rather than being an arbitrary stressor. missing_evidence spans both the retrieval and evidence layers because it removes support regardless of whether that support was ever retrieved.

This stage-anchored view is what justifies treating the six operators as a taxonomy rather than a convenience list: the four pipeline stages a RAG system executes between a corpus update and a served answer — retrieval, chunking, context assembly, and knowledge-state consistency — each have at least one dedicated operator, and no stage has more than two. `missing_evidence` and `hard_negative` probe whether a model correctly abstains when the retriever has failed outright — the evidence was deleted, moved, or never indexed, or only near-duplicate pages were retrieved. `conflict` probes whether a model can identify and prefer the correct passage when a stale or superseded document remains in the index alongside the current one, a common artifact of partial rollouts, and is the one operator that targets knowledge-state consistency rather than a retrieval- or chunking-time defect. `context_noise` and `evidence_position` probe tolerance to an over-broad or poorly-ranked retrieval window. `chunk_boundary` probes whether an indexing-time decision — where a chunk is split — affects answer

3.5 Worked Example

The taxonomy in Table 2 is easiest to read against one concrete case traced end to end: the same clean seed, unmodified, missing its evidence, and holding a conflicting claim, followed by what three production models actually did with each version. All examples below are real pipeline output from seed run pilot_stability_s42 (data/runs/pilot_stability_s42/), not constructed for illustration.

Clean seed (seed_000001). Question: “Where was Clinton Joseph Davisson born?” Gold answer: “Bloomington, Illinois.” Supporting sentence: “Clinton Joseph Davisson was born on October 22, 1881, in Bloomington, Illinois, the son of Joseph Davisson, an artisan from Ohio, and Mary Calvert, a schoolteacher from Pennsylvania.” All three evaluation models answer correctly on this clean version.

missing_evidence (low). The gold chunk is removed; the two retained context chunks mention Davisson’s schooling and Nobel Prize but never his birthplace. Expected behavior: abstain. All three models correctly output “I don’t know” rather than guessing.

conflict (low). The gold chunk is kept and a second, rewritten chunk is inserted asserting the same sentence with the location changed: “Clinton Joseph Davisson was born on October 22, 1881, in Cairo, the son of Joseph Davisson...” (alternate_answer = “Cairo”). Expected behavior: answer, preferring gold. The three models diverge on this single case: gpt-oss answers “Bloomington, Illinois” (correct); nova-pro abstains (“I don’t know”) despite gold being present; llama answers “Bloomington, Illinois, and Cairo” — hedging by naming both locations, which the LLM judge marks incorrect. This single case is illustrative, not representative of the aggregate per-model pattern in Table 12 (Section 5.7); it is included because it is a real, human-readable instance of the three qualitatively different ways a model can fail conflict — wrong abstention, answer-blending, and a clean correct pick.

This single seed is one of 200 clean seeds and roughly 1,900–2,000 derived failure cases per seeded run (Table 4); Section 5.7 reports the same missing_evidence / conflict comparison aggregated as accuracy and abstention rate, mean $\pm$ std, across all cases and all three seeds.

Table 3 extends the same idea to the remaining four operators with one compact, real example each, so a reader can see what each operator actually does to a context without reading the codebase.

Table 3. One real example per remaining Failure Operator (missing_evidence and conflict are traced above). Full context text and verification metadata for every case are in data/runs/pilot_stability_s42/6_final/failures/.

Operator	Example (real pipeline output, medium severity unless noted)
context_noise	Q: “Where was Clinton Joseph Davisson born?” — gold chunk kept, plus 4 unrelated distractor chunks added (e.g., a passage on academic conference lecture topics, one on a politician’s re-election). Expected: answer. Noise ratio is the severity parameter.
chunk_boundary	Same question — the supporting sentence is split mid-clause across chunk boundaries: “Clinton Joseph Davisson was born on October 22, 1881, in Bloomington, Illinois, the son” “is the initial wavelength,” (an unrelated chunk inserted between) “of Joseph Davisson, an artisan from Ohio...”. Expected: answer. Split depth is the severity parameter.
evidence_position	Same question — gold chunk is kept but relocated among 8 total context chunks (front / middle / end across severities) rather than left in its natural position. Expected: answer. Position is the severity parameter.
hard_negative	Q: “When did Pavel Alekseyevich Cherenkov die?” — gold chunk replaced entirely with topically related but non-answering passages about other Nobel-laureate physicists (Vitaly Ginzburg, Erwin Schrödinger). Expected: abstain. Negative count is the severity parameter.

4. Experimental Setup

Table 4. Pilot experimental configuration.

Item	Setting
Corpus	1,000 target English Wikipedia pages → 768 pages after filtering → 12,366 chunks (frozen for the stability experiment)
Categories	person, location, science/technology, historical event, organization/product
Candidates → seeds	1,000 QA candidates → 200 clean seeds per run
Context budget	8 chunks (identical for clean and failure conditions)
Difficulty quota	40% easy / 40% medium / 20% hard, enforced at generation
QA generation / validation model	Qwen2.5-7B-Instruct
Evaluation models	nova-pro, llama, and gpt-oss, on a shared evaluation gateway
Primary evaluation run	all three seeded runs (200 clean cases + $\approx$ 1,900–2,000 failure cases per run) $\times$ 3 models, reported as mean $\pm$ std
Stability protocol	M1 corpus frozen once; M2–M3 (generation, validation, injection) repeated with seeds {42, 123, 2026}

We report two families of experiments. The first, a stability study, freezes the M1 corpus once and repeats candidate generation, validation, and failure injection three times with different random seeds (42, 123, 2026), so that the automatic construction funnel and the human-audited quality of its output can be reported as a mean $\pm$ standard deviation rather than a single run. The second, a model evaluation, runs three evaluation models (nova-pro, llama, and gpt-oss) over the clean seeds and all injected failure cases from each of the three seeded runs, scoring exact-match, token-F1, and LLM-judge correctness, and separately tracking the abstention rate; per-model, per-condition results are reported as a mean $\pm$

standard deviation over the three seeds rather than from a single run. Human review is performed with exported CSV review sheets: one reviewer independently marks each clean seed as keep/reject to compute a human acceptance rate (HAR), and a stratified sample of failure cases per operator is independently marked as structurally valid or not. We note this pilot uses a single annotator; inter-annotator agreement is left to future work (Section 6.5).

5. Results

5.1 Pipeline Statistics

Across the three seeded runs on the frozen pilot corpus, the automatic construction funnel is stable: 1000.0 ± 0.0 QA candidates are generated per run, of which $99.8\% \pm 0.1\%$ pass schema validation, and $39.2\% \pm 0.2\%$ survive the full rule / judge / baseline funnel to become clean seeds (200.0 ± 0.0 per run by construction). Failure-injection structural verification passes at $97.5\% \pm 0.1\%$. The full breakdown by run is given in Table 6.

Table 5. Pipeline statistics, mean $\pm$ standard deviation over three seeded runs on the frozen pilot corpus.

Metric	Mean $\pm$ std
Candidate QA count	1000.0 ± 0.0
Schema success	$99.8\% \pm 0.1\%$
QA acceptance	$39.2\% \pm 0.2\%$
Clean-seed count	200.0 ± 0.0
Failure verification pass	$97.5\% \pm 0.1\%$
Human acceptance (HAR)	$66.0\% \pm 1.8\%$
Human failure validity	$96.8\% \pm 1.2\%$

5.2 Validation Funnel by Run

Table 6. Validation funnel and human audit outcomes, per seeded run.

Run ID	Accept %	Fail-verify pass %	HAR	Failure validity
pilot_stability_s42	39.3%	97.5%	66.5%	97.7% (299/306)
pilot_stability_s123	39.2%	97.6%	64.0%	95.4% (292/306)
pilot_stability_s2026	39.0%	97.5%	67.5%	97.4% (298/306)

Acceptance is intentionally low: the strict combination of the title_leak_in_question rule and a minimum-quality-score threshold of 0.85 filters aggressively by design, and the 20% yield (200 seeds from 1,000 candidates) is a configured target rather than an incidental result. After difficulty-quota-enforced regeneration, clean-seed difficulty mixes are aligned across runs at approximately 44–46% easy, 39–40% medium, and 15–16% hard.

5.3 Validation Component Contribution

Section 3.3 notes that validation is not a strict rule $\rightarrow$ judge $\rightarrow$ baseline funnel: judge and baseline run independently on rule-passing candidates rather than one gating the other. Table 7 quantifies each automatic signal’s contribution using the per-candidate records already stored in 5_validated/validation_results.jsonl for each of the three seeded runs (1,000 candidates each) — no re-generation or re-judging was required. For each signal, “Rejected candidates” counts every candidate whose stored rejection_reasons include that signal (rows can overlap, since a candidate may fail more than one check), and “Unique rejections” counts candidates whose rejection_reasons map to only that signal, isolating checks that are not redundant with the others.

Table 7. Validation component contribution, mean $\pm$ standard deviation over three seeded runs (1,000 candidates each).

Validation signal	Rejected candidates	Unique rejections	Typical reason
Rule checks	561.7 ± 6.4	56.0 ± 3.0	title leakage / containment / uniqueness
LLM Judge	31.3 ± 4.7	9.7 ± 1.5	unsupported / unclear / unanswerable
Baseline test	40.7 ± 4.9	2.7 ± 1.5	gold-chunk baseline incorrect
Quality threshold	537.3 ± 4.6	0.0	combined score below min_quality_score
Deduplication	2.7 ± 0.6	2.7 ± 0.6	near-duplicate question or fact

Rule checks account for the large majority of rejection volume and the most unique rejections (56.0 ± 3.0), consistent with rules being the first, cheapest filter. The LLM judge and baseline test each still contribute a nontrivial number of unique rejections (9.7 ± 1.5 and 2.7 ± 1.5 respectively) that no other signal would have caught — direct evidence that neither is redundant with the rule stage or with each other. The quality threshold rejects a large volume of candidates (537.3 ± 4.6) but essentially never rejects a candidate that no other signal has already flagged (0.0 unique), because it operates on a combined score that already reflects the rule/judge/baseline outcomes rather than being an independent check. This is a validation component contribution analysis, not a per-stage human-acceptance-rate (HAR) ablation: the human review CSVs cover only the final clean-seed set, not intermediate populations such as rule-only-passing or rule+judge-passing candidates, so a true rule-only HAR or rule+judge HAR would require drawing and annotating new stratified samples from those intermediate populations — no re-run of any LLM is needed, but new human annotation is, and this remains an open item in Section 6.5 rather than something the existing artifacts can answer directly.

5.4 Human Audit

Table 8. Human validation of clean seeds and injected failure cases, per run.

Run	Clean HAR	Failure sample validity
pilot_stability_s42	66.5% (133/200 keep)	97.7% (299/306; 6 operators × 51)
pilot_stability_s123	64.0% (128/200 keep)	95.4% (292/306; 6 operators × 51)
pilot_stability_s2026	67.5% (135/200 keep)	97.4% (298/306; 6 operators × 51)
Mean ± std	66.0% ± 1.8%	96.8% ± 1.2%

HAR should be read as the quality of the selected seed set, not of raw candidates, and at this scale it reveals a real gap rather than a formality: a low automatic acceptance rate ($\approx 39\%$) does not translate into a correspondingly high human-audited acceptance rate ($\approx 66\%$) — roughly a third of automatically accepted clean seeds are rejected on human inspection, so the automatic funnel is a necessary but not sufficient quality gate on its own. All three seeded runs now provide full six-operator stratified failure-validity coverage (51 cases $\times$ 6 operators = 306 per run), resolving the four-of-six coverage gap in the earlier 500-page / 100-seed pilot; human-audited failure validity remains high but is more variable across runs than clean-seed HAR (95.4–97.7%). Section 5.3 reports a validation component contribution analysis that attributes automatic rejection volume to each of the rule, judge, and baseline signals; Section 5.5 goes a step further and categorizes exactly what human reviewers caught among the automatically accepted candidates that were marked fix, which is what explains the acceptance-HAR gap itself. A true per-stage HAR ablation, isolating how much each automatic signal specifically contributes to that gap via new stratified human annotation on intermediate populations, remains an open item in Section 6.5, though now a lower priority given the explanation Section 5.5 already provides.

5.5 Human Fix-Reason Breakdown

The component contribution analysis in Section 5.3 explains why the automatic funnel rejects roughly 61% of candidates; it does not explain the further gap between automatic acceptance and human-audited HAR (Section 5.4). To close that gap, we categorize the 204 clean seeds across the three seeded runs (600 reviewed total) that human reviewers marked fix rather than keep, using the structured review-CSV flags (question_clear, answer_unique, needs_title, time_sensitive_ok) directly rather than parsing free-text reviewer notes, so the categorization is reproducible from the review artifacts alone. No seed was marked reject in any of the three runs: the automatic funnel’s answer-support checks appear sufficient to prevent that specific failure mode on their

own, so the entire human-audited quality gap is a fix-severity gap.

Table 9. Primary human fix reason for the 204 clean seeds marked fix across three seeded runs (600 reviewed; one primary reason per row, so percentages sum to 100%).

Fix reason	Count	Percent	Structured basis
Referential ambiguity (unresolved pronoun; needs page title)	146	71.6%	question_clear = no
Answer not unique	38	18.6%	answer_unique = no
Time-sensitive / stale-prone phrasing	20	9.8%	time_sensitive_ok = no

Referential ambiguity dominates and is not the noisy artifact it first appeared to be: every one of the 146 flagged questions contains an unresolved third-person pronoun or reference (he, she, they, it, their, ...), and this same 146 also accounts for essentially all of the needs_title = yes flags among fix rows — in this data, “needs the page title to be understandable” and “unclear because of an unresolved pronoun” are the same underlying phenomenon rather than two independent problems. The remaining 20 time-sensitive cases (e.g., “as of 2025” population figures, “sometimes known as...” framing) are phrasing that would go stale or stay ambiguous as the corpus is regenerated; about half of these also need the page title.

This breakdown also answers a question a reviewer is likely to ask: why rely on human review rather than training another LLM judge on the same signals? The validation pipeline’s own LLM judge already produces question_clear and answer_unique signals for some candidates (Section 3.3). Joining the clean-seed records back to the underlying validation_results.jsonl shows that, for the 274 of 600 clean seeds where the judge signal was actually recorded, it marked every one of them clear and every one of them unique — expected, since a negative judge signal would ordinarily have blocked acceptance. Yet on human re-inspection, 60 of those 274 judge-cleared seeds (21.9%) were found to have an unresolved pronoun or reference ambiguity, and 25 (9.1%) were found to have a non-unique answer. On precisely the two axes where the automatic judge already produces a signal, a meaningful share of its accept decisions do not hold up to human review. This supports treating human review as catching a distinct class of pragmatic and real-world-knowledge judgments — resolving what a pronoun refers to, recognizing that two people share a surname, anticipating that a phrase like “as of 2021” will go stale — rather than as a stricter re-run of the same check the judge already performs.

Table 10. Validation and review responsibilities across the pipeline: dominant failure mode caught by each stage.

Stage	Primary signal	Dominant failure mode(s) caught
Rule	title_leak_in_question, containment, uniqueness heuristics	Title leakage, duplicate/near-duplicate questions, structural containment failures
LLM Judge	answerability / faithfulness	Unsupported or unfaithful answers
Baseline	gold-chunk answerability	Not answerable from gold evidence alone
Human review	question_clear, answer_unique, time_sensitive_ok	Referential/pronoun ambiguity, non-unique answers, time-sensitive or stale-prone phrasing

Each stage therefore has a distinguishable area of responsibility rather than four stages screening for overlapping variants of the same problem — the strongest evidence available in this pilot that human review is a complementary rather than a redundant final stage of the pipeline.

5.6 Failure Operator Verification

Table 11. Failure-injection structural verification and human-audited validity, aggregated across seeds.

Check	Result
Automatic structural verification (leakage / label check)	97.5% $\pm$ 0.1% pass
Human-audited stratified validity (all 6 operators, all three seeds)	95.4–97.7% pass

5.7 Model $\times$ Operator Performance

Across the three seeded runs, mean clean-context accuracy is 0.83 ± 0.01 for nova-pro, 0.82 ± 0.01 for llama, and 0.85 ± 0.01 for gpt-oss, with aggregate failure-robustness scores of 0.78 ± 0.00 , 0.80 ± 0.00 , and 0.76 ± 0.01 respectively. Table 12 and Table 13 (and Figure 2) break this down by operator, reporting mean $\pm$ standard deviation over the three seeds rather than results from a single run.

Table 12. Model accuracy by condition, mean $\pm$ standard deviation over three seeded runs (Δ relative to each model's own mean clean accuracy).

Condition	nova-pro acc $\pm$ std (Δ)	llama acc $\pm$ std (Δ)	gpt-oss acc $\pm$ std (Δ)
clean	0.83 ± 0.01	0.82 ± 0.01	0.85 ± 0.01
missing_evidence	0.05 ± 0.02 (−0.78)	0.04 ± 0.01 (−0.78)	0.05 ± 0.02 (−0.79)
hard_negative	0.02 ± 0.01 (−0.81)	0.03 ± 0.01 (−0.79)	0.04 ± 0.02 (−0.81)
conflict	0.82 ± 0.03 (−0.01)	0.81 ± 0.02 (−0.01)	0.71 ± 0.07 (−0.14)
context_noise	0.93 ± 0.01 (+0.10)	0.95 ± 0.02 (+0.13)	0.96 ± 0.01 (+0.12)
chunk_boundary	0.92 ± 0.01 (+0.09)	0.95 ± 0.03 (+0.13)	0.91 ± 0.02 (+0.07)
evidence_position	0.95 ± 0.00 (+0.11)	0.96 ± 0.01 (+0.14)	0.97 ± 0.01 (+0.12)

Table 13. Model abstention rate by condition, mean $\pm$ standard deviation over three seeded runs.

Condition	nova-pro abst. $\pm$ std	llama abst. $\pm$ std	gpt-oss abst. $\pm$ std
missing_evidence	0.87 ± 0.02	0.89 ± 0.01	0.87 ± 0.02
hard_negative	0.93 ± 0.01	0.92 ± 0.01	0.93 ± 0.02
conflict	0.06 ± 0.01	0.01 ± 0.01	0.19 ± 0.05
context_noise	0.05 ± 0.00	0.01 ± 0.01	0.01 ± 0.01
chunk_boundary	0.06 ± 0.02	0.02 ± 0.02	0.06 ± 0.01
evidence_position	0.03 ± 0.00	0.01 ± 0.01	0.01 ± 0.00

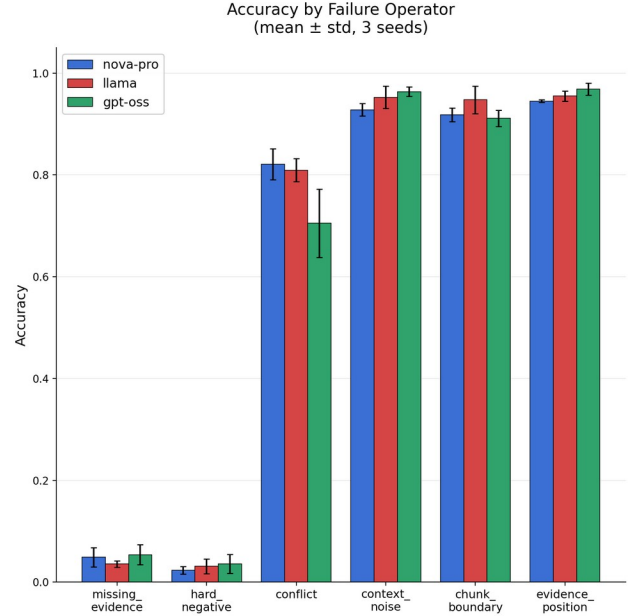


Figure 2. Mean accuracy by Failure Operator for nova-pro, llama, and gpt-oss, aggregated over three seeded runs. Absence-type operators (missing_evidence, hard_negative) collapse accuracy toward zero as models correctly abstain; context_noise, chunk_boundary, and evidence_position leave accuracy at or above the clean-context ceiling for all three models; conflict is the one condition where the three models clearly diverge, with gpt-oss dropping well below nova-pro and llama.

Five findings follow from Tables 12–13. First, absence operators work as intended across all three models: nova-pro, llama, and gpt-oss all abstain on 87–93% of missing_evidence and hard_negative cases rather than hallucinating an answer, which is the desired safety behavior when the retriever has failed outright. Second, context_noise, chunk_boundary, and evidence_position are weak stressors for all three production-grade models at the severities used in the pilot — accuracy on these conditions is consistently at or above each model's own clean-context accuracy, a useful negative result that aggregate accuracy alone would have hidden by averaging a near-zero absence-operator score against a near-ceiling positional-robustness score. Third, conflict is the one operator where the three models diverge sharply:

nova-pro and llama both take a small, consistent hit (≈ 1 accuracy point, 1–6% abstention) while gold evidence remains present in context, but gpt-oss drops 14 accuracy points and abstains on 19% of conflict cases — roughly triple the abstention rate of the other two models — indicating gpt-oss is materially more sensitive to a contradictory alternate claim sitting alongside the correct evidence. Fourth, this divergence means the cross-model ranking of operator difficulty is consistent for five of six operators but not for conflict: all three models find `missing_evidence` and `hard_negative` hardest and `context_noise`, `chunk_boundary`, and `evidence_position` easiest, but only gpt-oss finds conflict comparably hard to the absence operators in terms of abstention behavior. Fifth, and directly addressing RQ2 (do models differ in their sensitivity to specific operators), adding a third, architecturally distinct evaluation model shows that the operator taxonomy's difficulty ordering generalizes broadly across model families, while also surfacing genuine, operator-specific model differences — exactly the kind of diagnostic signal an aggregate accuracy score would hide, and the core motivation for treating failure operators as first-class, separately reported conditions rather than folding them into a single benchmark score.

6. Discussion

6.1 Why Automated Generation Works

Rule checks, the LLM judge, and the baseline-answer test are not links in a single sequential chain — the judge and baseline run independently on rule-passing candidates — but they are still effective in combination because each targets a different, largely independent failure mode in generated QA. The rule stage is cheap and catches structural shortcuts — most importantly, questions that leak the answer entity through the page title, which a purely semantic judge might not flag because the question still reads as fluent and on-topic. The judge signal catches faithfulness failures — a question whose stated answer is not actually supported by the cited sentence — that a rule cannot detect because it requires comparing meaning, not surface form. The baseline signal catches a third, distinct failure mode: a question that is faithful and non-leaking but is nonetheless too ambiguous or too dependent on external context for even a capable reference model to answer correctly from the gold chunk alone. This is no longer only a qualitative claim: the validation component contribution analysis in Section 5.3 shows that, among automatically rejected candidates, rules produce the most unique rejections (56.0 ± 3.0 per run) but the judge (9.7 ± 1.5) and baseline (2.7 ± 1.5) each still catch a nontrivial number of candidates that no other signal would have

flagged — direct evidence that neither is redundant with the rule stage or with each other. Combined, the automatic signals are consistent with the pilot's low raw acceptance rate ($\approx 39\%$): the funnel is discarding the clear majority of candidates that would otherwise inflate a benchmark with unanswerable, leaking, or unfaithful items. That said, the funnel is evidently not sufficient on its own — human-audited acceptance on the retained set is only $\approx 66\%$, meaning roughly a third of automatically accepted candidates still fail human review, and Section 5.3's component analysis attributes rejection volume among automatically rejected candidates but says nothing about the quality of the automatically accepted set, so it cannot by itself explain the acceptance-HAR gap. Section 5.5 closes that gap directly: 71.6% of human fix decisions trace to unresolved pronoun or reference ambiguity, 18.6% to non-unique answers, and 9.8% to time-sensitive or stale-prone phrasing, and joining these seeds back to the validation records shows the judge's own clarity and uniqueness signals were never negative on any of them. The automatic signals are not merely stricter versions of what a human catches — they check for a different class of failure (surface leakage, answer support, gold-chunk answerability) than the pragmatic and real-world-knowledge judgments a human reviewer makes (pronoun resolution, answer uniqueness, temporal staleness). This is evidence against training another LLM judge as a wholesale substitute for human review on these specific axes, and in favor of adding pronoun-resolution and staleness checks as new automatic signals rather than treating the existing judge as redundant with human review. A true per-stage HAR ablation (rule-only vs. rule+judge vs. rule+judge+baseline, each independently human-reviewed on new stratified samples) would still be the rigorous way to attribute the gap to a specific automatic stage rather than to the categories of human judgment identified here, and remains an open item in Section 6.5, though now a lower priority given the first-order explanation the fix-reason breakdown already provides.

6.2 Failure Insights

`hard_negative` is a harder condition to engineer correctly than `context_noise` because a near-miss passage is, by construction, close enough to the correct topic that a model can be misled into answering from it confidently rather than abstaining — the failure is not just “more text to search through” as in `context_noise`, but text that plausibly resembles the answer. In the pilot, both operators produce a similar abstention-dominant behavior (Table 12), but that outcome should not be read as evidence the operators are interchangeable: `context_noise`

keeps the gold chunk in context and is expected to be answerable, while `hard_negative` omits it and is expected to trigger abstention, so their similar abstention rates in fact reflect the models correctly distinguishing between an answerable-but-noisy condition (low abstention observed for `context_noise`) and an unanswerable one (high abstention observed for `hard_negative`), exactly as their respective expected-behavior labels specify. The larger and more actionable insight is the conflict result: gold evidence remains present in context, yet nova-pro and llama still lose roughly 1 accuracy point while gpt-oss loses 14, indicating that ordinary context-window robustness is not sufficient once an enterprise index contains a stale but not-yet-purged document, that this is a distinct engineering problem from either retrieval recall or context length, and that a team's exposure to this specific failure mode is model-dependent rather than a fixed property of the RAG pipeline — a regression suite that only tracked aggregate accuracy, or that evaluated a single model, would have missed how differently gpt-oss behaves under this one condition.

6.3 Engineering Implications

Sections 5.7 and 6.2 report which operators hurt accuracy; Table 14 restates the same six results as an engineering question — if a team saw this operator's degradation in their own regression report, which system component should they look at first? The mapping follows directly from what each operator manipulates (Table 2, Section 3.4), not from a separate analysis.

Table 14. Engineering implications: the system component a regression in each operator's score most directly implicates.

Operator	What degraded (Section 5.7)	Component to check first
missing_evidence	All three models correctly abstain — the desired behavior when evidence is absent	Retriever recall / indexing coverage: in production, frequent missing_evidence-like abstention signals real documents are not being retrieved at all
hard_negative	All three models correctly abstain under topical near-miss distractors	Retriever precision / re-ranking: the retriever needs to discriminate near-miss passages from gold, not just return topically relevant results
conflict	The one operator with a real accuracy hit, and the one where models diverge sharply (Table 12–13)	Document lifecycle / version control: stale or superseded documents must be purged or flagged at index time, not left for the model to adjudicate at query time
context_noise	Near clean-context ceiling for all three models at pilot severities	Retrieval window / top-k: current top-k budgets are not yet the bottleneck, but should be re-tested if the operator is run at higher severities
chunk_boundary	Near clean-context ceiling for all three models	Chunking strategy: current split-depth choices are not yet the bottleneck, but this is the check to re-run after any chunk-size change
evidence_position	Near clean-context ceiling for all three models	Prompt structure / context ordering: current position handling is not yet the bottleneck, but this is the check to re-

Operator	What degraded (Section 5.7)	Component to check first
		run after any prompt-template change

Read this way, the six operators are not just failure categories but a checklist against the four RAG pipeline stages in Figure 1: a conflict regression points at index freshness, a missing_evidence or hard_negative regression points at the retriever, and a context_noise / chunk_boundary / evidence_position regression — currently the least sensitive of the six at pilot severities — points at retrieval-window, chunking, or prompt configuration respectively. Section 6.4 describes how this per-operator report is wired into an actual deployment gate.

6.4 Enterprise Regression Testing

The practical motivation for this framework is the deployment loop described in Section 1: a knowledge base changes, the RAG index is rebuilt, and a team needs to know, before shipping, whether retrieval robustness regressed and against which failure mode. RAGFailBench is designed to sit inside that loop rather than beside it, and Figure 3 makes the loop concrete. When source documents are updated — through a connector to Confluence, SharePoint, or a documentation repository, in place of the Wikipedia source adapter used in this pilot — the corpus-construction stage (Section 3.1) can be rerun to produce a new chunk set; the QA-generation and validation stages (Sections 3.2–3.3) regenerate a fresh, stratified clean-seed pool from that updated corpus rather than reusing a frozen question set that may now reference deleted or moved content; the failure-injection stage (Section 3.4) regenerates all six typed failure conditions against the new seeds; and the evaluation stage produces a per-operator robustness report that a release process can gate on — for example, blocking a deployment if the missing_evidence abstention rate drops below a threshold, or if the conflict accuracy delta widens beyond the pilot baseline.

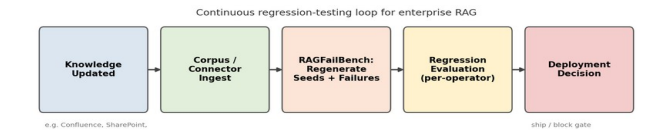


Figure 3. The RAGFailBench regression-testing loop: a knowledge-base update triggers corpus re-ingestion, automatic benchmark regeneration (seeds and failure cases), per-operator regression evaluation, and a gated deployment decision.

We treat this loop — knowledge update → automatic benchmark regeneration → regression evaluation → deployment decision — as the paper's primary practical

contribution (C3), and as the framing that most distinguishes this work from prior RAG benchmarks (Table 1): RAGAS, ARES, RGB, RAGBench, and CRUD-RAG each assume a benchmark that already exists, whereas RAGFailBench treats benchmark regeneration as a first-class, automatable step in the same pipeline that regenerates the RAG index itself. This reframes RAG evaluation from a one-time academic leaderboard entry into a recurring, CI-triggerable check, in the same spirit as regression-test selection in traditional software engineering [47,48] but applied to a benchmark dataset that is itself regenerated rather than fixed. The pilot in this paper validates the construction and diagnosis stages of this loop on an open, easily-inspected corpus (Wikipedia); Section 6.5 discusses what remains to validate the connector-ingest stage on genuine enterprise document formats.

6.5 Limitations

The current pilot has several limitations that bound the claims in this paper. Most importantly, there is a roughly 34-point gap between automatic acceptance ($\approx 39\%$) and human-audited HAR ($\approx 66\%$) on the retained set (Section 5.4): the automatic funnel is not, by itself, a reliable proxy for final benchmark quality, and every seed set used for the results reported here has depended on a human-review pass to reach its stated quality level. The corpus is drawn from the live MediaWiki API rather than a pinned revision or archival dump, so exact corpus statistics are not guaranteed to be reproducible bit-for-bit by a third party rerunning the pipeline today; a pinned-revision ingest path is planned before any camera-ready data claim. `hard_negative` selects near-miss passages using lexical overlap rather than a learned dense-retrieval score, which may understate how a production dense retriever would actually surface near-misses. The conflict operator's alternate claims are template- or rewrite-based and may read as stylistically unnatural compared to a genuinely stale enterprise document. The weak drops observed for `context_noise`, `chunk_boundary`, and `evidence_position` at pilot severities may indicate either that these operators are under-stressing the models at their current parameter settings or that the three evaluated models are simply strong along these particular axes; we do not yet have evidence to distinguish these explanations, and testing weaker models or higher severities is necessary before concluding an operator is "solved." Pilot human review used a single annotator, so inter-annotator agreement has not yet been measured — this is a more pressing gap now than it would be with a high HAR, since a single reviewer's judgment calls are directly responsible for a third of automatically accepted

seeds being discarded. The paper's central quantitative claims (Section 6.1, Section 6.2) now draw on two analyses computed directly from existing artifacts: a validation component contribution analysis (Section 5.3) that attributes automatic rejection volume to each of the rule, judge, baseline, quality-threshold, and deduplication signals, and a categorized breakdown of human fix reasons on the final clean-seed set (Section 5.5) that attributes the acceptance-HAR gap to referential ambiguity, non-unique answers, and time-sensitive phrasing. What remains open is a true per-stage human-acceptance-rate (HAR) ablation (rule-only vs. rule+judge vs. rule+judge+baseline, each independently human-reviewed on new stratified samples drawn from intermediate populations) that would attribute the gap to a specific automatic stage rather than to a category of human judgment; this requires new stratified human annotation rather than a rerun of any model, since neither existing analysis measures what a human reviewer would have decided about candidates that passed only a subset of the automatic signals. Given that the fix-reason breakdown already provides a first-order explanation for the acceptance-HAR gap, we now treat the per-stage HAR ablation as a lower-priority addition than it was before that analysis existed, though it remains the most rigorous way to confirm the explanation. Finally, Wikipedia is a deliberate pilot-corpus choice for this paper, not a Wikipedia-specific framework: every pipeline stage (source ingestion, QA generation, validation, failure injection, regeneration) operates on chunked text with provenance metadata, and Wikipedia was selected because it is open and independently verifiable, letting a reviewer or re-implementer check every reported number without a data-access agreement — a property no enterprise corpus offers. The pilot is restricted to single-hop, Wikipedia-sourced facts in English; extending the corpus-construction stage to multi-hop questions and to genuinely private enterprise document formats (Confluence exports, SharePoint libraries, PDF policy documents) is future work and is architecturally supported by the pluggable sources module (Section 3.1) but not yet evaluated end-to-end. More broadly, the construction-validation-injection-regeneration loop introduced here for generation and answer robustness is not specific to generation: retriever evaluation (does the retriever itself degrade under the same corpus changes that motivate conflict and missing_evidence), reranker evaluation (does a reranking stage recover the accuracy that `context_noise` and `evidence_position` take away), and prompt evaluation (how much of the `context_noise` / `chunk_boundary` / `evidence_position` robustness observed here is attributable to prompt template rather than model

capability) are natural, directly buildable extensions of the same regeneration loop rather than separate benchmarks.

7. Conclusion

We presented RAGFailBench, a framework built around one idea: automatically regenerate a failure-aware benchmark whenever the knowledge base changes, rather than release a benchmark once and let it go stale. This reframes RAG evaluation as controllable failure injection combined with matched-budget diagnosis, rather than a static leaderboard score. Across three seeded runs on a frozen 1,000-page pilot corpus, the automatic construction pipeline is reproducibly stable (QA acceptance $39.2\% \pm 0.2\%$; failure-verification pass $97.5\% \pm 0.1\%$) and its output is human-audited, with a clean-seed HAR of $66.0\% \pm 1.8\%$ and stratified failure validity of $96.8\% \pm 1.2\%$ — both stable across seeds, though the roughly 34-point gap between automatic acceptance and human-audited HAR (Section 5.4) shows that human review remains a necessary, not merely confirmatory, stage of the pipeline: categorizing what reviewers actually caught (Section 5.5) shows they are resolving pronoun and reference ambiguity, answer uniqueness, and time-sensitive phrasing — a different class of judgment than the surface leakage, support, and answerability checks the automatic rule/judge/baseline signals already perform, not a stricter version of the same check. Evaluating three production-grade models — nova-pro, llama, and gpt-oss — exposes clear, operator-specific behavior that an aggregate accuracy score would have hidden: all three models correctly abstain when evidence is absent and remain close to their clean-context ceiling under context noise, chunk-boundary splitting, and evidence-position perturbation, while conflict separates the models sharply, with gpt-oss taking a substantially larger accuracy and abstention hit than nova-pro or llama. This combination of a consistent cross-model difficulty ordering and a genuine, operator-specific model difference is exactly the diagnostic signal a single aggregate score would erase, and it is the generality argument for treating RAGFailBench as a model-agnostic diagnostic framework rather than a two-model case study. Because the pipeline is regenerable end-to-end — from a live corpus, through validated QA generation, through a stage-anchored taxonomy of six failure operators (Figure 1), to a per-operator robustness report — RAGFailBench is designed to function as a continuous regression-testing artifact for enterprise RAG deployments (Figure 3), connecting knowledge-base updates directly to deployment decisions, rather than as a one-time benchmark release. We release the code, configuration

files, and regenerable artifacts at <https://github.com/taixingbi/RAGFailBench> so that others can ablate operators and extend the corpus-construction stage to enterprise document formats, rather than only download a frozen question set.

References

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in Proc. NeurIPS, 2020.
- [2] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, “Retrieval-Augmented Generation for Large Language Models: A Survey,” arXiv:2312.10997, 2024.
- [3] P. Zhao, H. Zhang, Q. Yu, Z. Wang, Y. Geng, F. Fu, L. Yang, W. Zhang, and B. Cui, “Retrieval-Augmented Generation for AI-Generated Content: A Survey,” arXiv:2402.19473, 2024.
- [4] Y. Huang and J. Huang, “A Survey on Retrieval-Augmented Text Generation for Large Language Models,” arXiv:2404.10981, 2024.
- [5] O. Ram, Y. Levine, I. Dalmedigos, D. Muhlgay, A. Shashua, K. Leyton-Brown, and Y. Shoham, “In-Context Retrieval-Augmented Language Models,” Trans. Assoc. Comput. Linguist. (TACL), 2023.
- [6] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAS: Automated Evaluation of Retrieval Augmented Generation,” in Proc. EACL (System Demonstrations), 2024.
- [7] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, “ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems,” in Proc. NAACL, 2024.
- [8] J. Chen, H. Lin, X. Han, and L. Sun, “Benchmarking Large Language Models in Retrieval-Augmented Generation,” in Proc. AACL, 2024.
- [9] R. Friel, M. Belyi, and A. Sanyal, “RAGBench: Explainable Benchmark for Retrieval-Augmented Generation Systems,” arXiv:2407.11005, 2024.
- [10] Y. Lyu, Z. Li, S. Niu, F. Xiong, B. Tang, et al., “CRUD-RAG: A Comprehensive Chinese Benchmark for Retrieval-Augmented Generation of Large Language Models,” arXiv:2401.17043, 2024.
- [11] National Institute of Standards and Technology, “TREC Retrieval-Augmented Generation (RAG) Track,” Text REtrieval Conference (TREC), 2024.
- [12] H. Yen, T. Gao, M. Hou, K. Ding, D. Fleischer, P. Izsak, M. Wasserblat, and D. Chen, “HELMET: How to Evaluate Long-Context Language Models Effectively and Thoroughly,” in Proc. ICLR, 2025.
- [13] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, “LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding,” arXiv:2308.14508, 2023.
- [14] X. Zhang, Y. Chen, S. Hu, Z. Xu, J. Chen, M. Hao, X. Han, Z. Thai, S. Wang, and Z. Liu, et al., “ ∞ Bench: Extending Long Context Evaluation Beyond 100K Tokens,” in Proc. ACL, 2024.
- [15] U. Shaham, M. Ivgi, A. Efrat, J. Berant, and O. Levy, “ZeroSCROLLS: A Zero-Shot Benchmark for Long Text Understanding,” in Findings of EMNLP, 2023.
- [16] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ Questions for Machine Comprehension of Text,” in Proc. EMNLP, 2016.
- [17] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, K. Toutanova, L. Jones, M. Kelcey, M. Chang, A. Dai, J. Uszkoreit, Q. Le, and S. Petrov, “Natural Questions: A Benchmark for Question Answering Research,” Trans. Assoc. Comput. Linguist. (TACL), vol. 7, pp. 453–466, 2019.
- [18] M. Joshi, E. Choi, D. S. Weld, and L. Zettlemoyer, “TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension,” in Proc. ACL, 2017.
- [19] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, “HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering,” in Proc. EMNLP, 2018.
- [20] F. Petroni, A. Piktus, A. Fan, P. Lewis, M. Yazdani, N. De Cao, J. Thorne, Y. Jernite, V. Karpukhin, J. Maillard, V. Plachouras, T. Rocktäschel, and S. Riedel, “KILT: a Benchmark for Knowledge Intensive Language Tasks,” in Proc. NAACL, 2021.
- [21] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” Found. Trends Inf. Retr., vol. 3, no. 4, pp. 333–389, 2009.
- [22] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, “Dense Passage Retrieval for Open-Domain Question Answering,” in Proc. EMNLP, 2020.
- [23] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, “Unsupervised Dense Information Retrieval with Contrastive Learning,” Trans. Mach. Learn. Res. (TMLR), 2022.
- [24] O. Khattab and M. Zaharia, “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT,” in Proc. SIGIR, 2020.
- [25] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, and M. Zaharia, “ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction,” in Proc. NAACL, 2022.
- [26] T. Formal, B. Piwowarski, and S. Clinchant, “SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking,” in Proc. SIGIR, 2021.

- [27] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, “C-Pack: Packaged Resources To Advance General Chinese Embedding,” arXiv:2309.07597, 2023.
- [28] BAAI (Beijing Academy of Artificial Intelligence), “BGE Reranker: Cross-Encoder Reranking Model,” FlagEmbedding repository and model card, 2023–2024.
- [29] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, “BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models,” in Proc. NeurIPS Datasets and Benchmarks Track, 2021.
- [30] P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. Soylu, M. Yasunaga, et al., “Holistic Evaluation of Language Models,” arXiv:2211.09110, 2022.
- [31] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C. Wu, et al., “MLPerf Inference Benchmark,” in Proc. ISCA, 2020.
- [32] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, “MTEB: Massive Text Embedding Benchmark,” in Proc. EACL, 2023.
- [33] E. Beeching, C. Fourrier, N. Habib, S. Han, N. Lambert, N. Rajani, O. Sanseviero, L. Tunstall, and T. Wolf, “Open LLM Leaderboard,” Hugging Face, 2023.
- [34] Y. Wang, Y. Kordi, S. Mishra, A. Liu, N. A. Smith, D. Khashabi, and H. Hajishirzi, “Self-Instruct: Aligning Language Models with Self-Generated Instructions,” in Proc. ACL, 2023.
- [35] C. Xu, Q. Sun, K. Zheng, X. Geng, P. Zhao, J. Feng, C. Tao, and D. Jiang, “WizardLM: Empowering Large Language Models to Follow Complex Instructions,” in Proc. ICLR, 2024.
- [36] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, “Alpaca: A Strong, Replicable Instruction-Following Model,” Stanford CRFM, 2023.
- [37] N. Ding, Y. Chen, B. Xu, Y. Qin, Z. Zheng, S. Hu, Z. Liu, M. Sun, and B. Zhou, “Enhancing Chat Language Models by Scaling High-quality Instructional Conversations,” in Proc. EMNLP, 2023.
- [38] C. Xu, Q. Sun, K. Zheng, X. Geng, P. Zhao, J. Feng, C. Tao, and D. Jiang, “WizardLM: Empowering Large Language Models to Follow Complex Instructions,” arXiv:2304.12244, 2023.
- [39] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, “G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment,” in Proc. EMNLP, 2023.
- [40] P. Manakul, A. Liusie, and M. Gales, “SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models,” in Proc. EMNLP, 2023.
- [41] S. Lin, J. Hilton, and O. Evans, “TruthfulQA: Measuring How Models Mimic Human Falsehoods,” in Proc. ACL, 2022.
- [42] J. Li, X. Cheng, W. X. Zhao, J. Nie, and J. Wen, “HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models,” in Proc. EMNLP, 2023.
- [43] LangChain, Inc., “LangSmith: Evaluation and Observability Platform for LLM Applications,” Product documentation, docs.smith.langchain.com, 2024.
- [44] OpenAI, “Evals: A Framework for Evaluating LLMs and LLM Systems,” GitHub repository, github.com/openai/evals, 2023.
- [45] Confident AI, “DeepEval: The Open-Source LLM Evaluation Framework,” GitHub repository, github.com/confident-ai/deepeval, 2024.
- [46] Promptfoo, “Open-Source LLM Testing and Evaluation Tool,” GitHub repository, github.com/promptfoo/promptfoo, 2024.
- [47] M. Fowler, “Continuous Integration,” martinowler.com, 2006.
- [48] G. Rothmel and M. J. Harrold, “Analyzing Regression Test Selection Techniques,” IEEE Trans. Softw. Eng., vol. 22, no. 8, pp. 529–551, 1996.
- [49] B. Fitzgerald and K.-J. Stol, “Continuous Software Engineering: A Roadmap and Agenda,” J. Syst. Softw., vol. 123, pp. 176–189, 2017.
- [50] Wikimedia Foundation, “Wikipedia: The Free Encyclopedia,” <https://www.wikipedia.org>.
- [51] Wikimedia Foundation, “MediaWiki API Documentation,” https://www.mediawiki.org/wiki/API:Main_page.
- [52] S. Auer, C. Bizer, G. Kobilarov, J. Lehmann, R. Cyganiak, and Z. Ives, “DBpedia: A Nucleus for a Web of Open Data,” in Proc. ISWC, 2007.